

Module #5

Structures Conditionnelles

1. La structure conditionnelle "switch"	1
2. La boucle "do while"	4
3. La structure "else if"	5
3.1. Introduction.....	5
3.2. Différence	6
3.3. Avantages	7
3.4. Risques et inconvénients	9
3.5. Ce qu'il faut retenir:.....	11

1. La structure conditionnelle "switch".

Le langage C présente une structure conditionnelle particulière, le **switch**. Cette structure est particulière dans le sens où elle permet de comparer une variable à plusieurs valeurs entières.

```
switch(Variable_A_Test)
{
    case Valeur1: Instructions à exécuter si la variable vaut Valeur1
        break;

    case Valeur2: Instructions à exécuter si la variable vaut Valeur2
        break;

    default: Instructions si la variable différente de Valeur1 et Valeur2
        break;
}
```

- Une structure switch peut avoir autant de case que vous le souhaitez.
- Le cas "default" est optionnel.
Si vous le mettez, les instructions lui correspondant seront exécutées si la variable ne vaut aucune des valeurs précisées dans les autres cas.
Si vous ne le mettez pas et que la variable est différente des valeurs précisées dans les autres cas, rien ne se passera.
- Il est important de préciser que le switch permet de comparer une variable qu'à des valeurs ENTIERES! Il est impossible d'utiliser cette structure conditionnelle pour comparer une variable à un point flottant, par exemple!

Et voici un exemple d'utilisation de la structure conditionnelle switch, sans le cas default:

```
UI iVarTest = 10;

switch(iVarTest)
{
    case 5:
        printf("iVarTest vaut 5\n");
        break;

    case 10:
        printf("iVarTest vaut 10\n");
        break;

    case 15:
        printf("iVarTest vaut 15\n");
        break;
}
```

Étant donné que la variable `uiVarTest` vaut 10, et que l'on a un cas qui correspond à cette valeur, on affichera le message: "uiVarTest vaut 10".

Note : Pour l'algo, on va utiliser une syntaxe semblable :

```
switch(variable)
    case 1:
        break.
FIN switch.
```

À présent, si la variable ne correspond à aucune des valeurs proposées, toujours sans cas default :

```
int iTest = 20;

switch(iTest)
{
    case 5:
        printf("iTest vaut 5\n");
        break;

    case 10:
        printf("iTest vaut 10\n");
        break;
}
```

Ici, `iTest` vaut 20, mais aucun cas correspondant à cette valeur. On n'affichera donc rien à l'écran.

Pour pallier à tous les cas non traités individuellement, il est possible d'utiliser une option par défaut, qui s'écrit de la manière suivante :

```
int iTest = 20;

switch(iTest )
{
    case 5:
        printf("iTest vaut 5\n");
        break;

    case 10:
        printf("iTest vaut 10\n");
        break;

    default:
        printf("iTest ne vaut ni 5 ni 10\n");
        break;
}
```

Ici encore, aucun cas ne correspond de manière précise à la valeur 20. Puisque l'on a un cas default, c'est donc celui-ci qui sera exécuté, et on affichera le message: "iTest ne vaut ni 5 ni 10".

Notez que l'instruction `break` à la fin de chaque cas est optionnelle.

Le `break` permet de quitter une structure de contrôle.

Donc, quand on le met à la fin d'un "case", ça termine le "case" et donc le "switch".

Si on ne le met pas, il se passera quelque chose expliqué dans l'exemple suivant:

```
int iPasBreak = 5;
```

```
switch(iPasBreak)
{
    case 5:
        printf("iPasBreak vaut 5\n");
        // Volontairement, on omet le break !

    case 10:
        printf("iPasBreak vaut 10\n");
        break;

    case 15:
        printf("iPasBreak vaut 15\n");
        break;
}
```

Qu'est-ce qui se passe ici ?

La variable "iPasBreak" vaut 5. On entrera donc dans le cas correspondant, et on affichera le message "iPasBreak vaut 5". Puisqu'on n'a pas d'instruction `break` à la fin de ce cas, on ne quittera pas la structure de contrôle et on continuera à exécuter les instructions du "case: 10". On affichera donc le message "iPasBreak vaut 10" ! Une fois ceci fait, on parviendra à une instruction `break`, et on quittera la structure "switch".

Il est parfois utile de ne pas mettre l'instruction `break`. Mais, souvent, en particulier pour les débutants, c'est un oubli qui entraîne de drôles de résultats à l'exécution du programme ! Soyez prudents.

Exemple où le `break` est omis de façon intentionnel.

```
cTouche = getkey();
switch(cTouche)
{
    case 'a':
    case 'A':
        // Instructions a faire si 'a' ou 'A' est entrée au clavier.
        break;

    case 'b':
    case 'B':
        // Instructions a faire si 'b' ou 'B' est entrée au clavier.
        break;

    ...
}
```

Comme on peut le voir, la structure de contrôle "switch" sera utile pour la lecture d'un clavier.

2. La boucle "do while".

La structure de boucle, que nous allons maintenant étudier, est "do...while", que l'on pourrait, en français, appeler "FAIRE... TANT QUE".

Ici encore, les instructions constituant la boucle sont exécutées tant que la condition de boucle est vraie. Cela dit, contrairement à while, avec do...while, la condition est évaluée à la fin de la boucle ; cela signifie que les instructions correspondant au corps de la structure de contrôle seront toujours exécutées au moins une fois, même si la condition est toujours fausse !

La syntaxe correspondant à cette structure répétitive est la suivante :

```
do
{
    Instruction à exécuter tant que la condition est vraie.
}while( Condition ); //
```

Prenons un exemple, qui affiche les valeurs décimales et hexadécimales du code ASCII des touches que nous entrons au clavier, en utilisant une boucle itérative de la forme **do...while**.

On quitte la boucle une fois que nous entrons la touche 'ESC'

```
#define ESC 27
do
{
    a = getchar();
    printf("%c = %bu = %bX\n", a, a, a);
}while(a != ESC);
```

Pour obtenir le même résultat avec une boucle **while**, nous devrions ajouter une instruction avant d'entrer dans la boucle pour s'assurer d'y entrer.

```
#define ESC 27
a = 0;
while(a != ESC)
{
    a = getchar();
    printf("%c = %bu = %bX\n", a, a, a);
}
```

Ainsi la boucle **do...while** garantie que les instructions ci trouvant seront exécuté au moins une fois.

Dans l'exemple suivant, nous pouvons constater que l'on entre dans la boucle, alors que la condition n'est pas vraie. Ce qui nous montre bien que la condition de boucle est testée en fin de boucle, et que les instructions de la boucle sont toujours exécutées, au moins une fois.

```
a = 0;
do
{
    printf("On est dans la boucle");
}while(a > 10);
```

Certes, il sera toujours possible d'utiliser le **while** au lieu du **do...while**, mais dans le but de simplifier le code ou d'en améliorer la lisibilité et la compréhension et dans les occasions où ce comportement correspond à ce que l'on recherche, il sera plus intéressant d'utiliser un **do...while** au lieu d'un **while** !

3. La structure "else if"

3.1 Introduction

La structure conditionnelle "else if" est une extension du "if" et permet d'exécuter différentes actions en fonction de plusieurs conditions.

La structure "else if" est utilisée pour gérer des cas multiples dans un programme. Elle permet d'améliorer la lisibilité et l'efficacité du code.

Syntaxe générale

Dans la plupart des langages, la syntaxe d'un "if", suivi d'un "else if", puis d'un "else", est la suivante :

```
if (condition1)
{
    // Code exécuté si condition1 est vraie
}
else if (condition2)
{
    // Code exécuté si condition1 est fausse et condition2 est vraie
}
else
{
    // Code exécuté si aucune des conditions précédentes n'est vraie
}
```

Exemple:

```
int main()
{
    int nombre = 10;

    if (nombre > 10)
    {
        printf("Le nombre est supérieur à 10.\n");
    }
    else if (nombre == 10)
    {
        printf("Le nombre est exactement 10.\n");
    }
    else
    {
        printf("Le nombre est inférieur à 10.\n");
    }
}
```

3.2 Différence

La différence entre "if" , "else if" , et "else" réside dans leur rôle et leur utilisation dans une structure conditionnelle :

1. **if** : Il sert à tester une première condition. Si cette condition est vraie, le bloc de code associé s'exécute, et les autres blocs else if ou else ne seront pas évalués.
2. **else if** : Il est utilisé pour tester une autre condition si la première (if) est fausse. Il peut y avoir plusieurs else if dans une même structure conditionnelle.
3. **else** : Il est utilisé comme une clause de secours qui s'exécute si aucune des conditions précédentes (if et else if) n'est vraie.

Différences clés :

- **if seul** : Permet de vérifier une condition unique.
- **if + else** : Permet d'avoir une alternative si la condition de if est fausse.
- **if + else if + else** : Permet de gérer plusieurs cas possibles.

3.3 Avantages.

L'utilisation de "else if" par rapport à une série de "if" séparés présente plusieurs **avantages** en termes de lisibilité, d'optimisation et de logique. Il est parfois préférable d'utiliser "else if" plutôt qu'une simple succession de "if" et "else".

Amélioration des performances

- **Avec else if** : Dès qu'une condition est **vraie**, les autres ne sont pas évaluées. Cela évite des tests inutiles et accélère l'exécution.
- **Avec plusieurs if séparés** : Toutes les conditions sont testées même si l'une d'elles est déjà satisfaite, ce qui peut ralentir le programme.

- Exemple inefficace avec plusieurs if :

```
if (age < 18)
{   printf("Mineur\n");   }
if (age >= 18 && age < 25)      // Ce test est inutile si le premier `if` était vrai
{   printf("Jeune adulte\n");   }
if (age >= 25)                  // Encore un test inutile
{   printf("Adulte\n");   }
```

- Solution optimisée avec else if :

```
if (age < 18)
{   printf("Mineur\n");   }
else if (age < 25)
{   printf("Jeune adulte\n"); }
else
{   printf("Adulte\n");   }
```


Meilleure lisibilité et organisation du code

- **else if** permet d'organiser clairement les différentes possibilités en une seule structure logique.
- Avec plusieurs if séparés, le programme devient plus difficile à lire et à comprendre.

Exemple avec else if (facile à comprendre) :

```
if (score >= 90)      {   printf("Excellent\n"); }
else if (score >= 75) {   printf("Bien\n");      }
else if (score >= 50) {   printf("Passable\n"); }
else                  {   printf("Échec\n");     }
```

Sans "else if" le code serait plus répétitif et plus dur à suivre.

Réduction des erreurs logiques

Avec plusieurs "if" séparés, on peut **par inadvertance** exécuter plusieurs blocs de code au lieu d'un seul.

"else if" garantit **qu'un seul bloc** est exécuté.

Problème avec plusieurs if :

```
int score = 85;
if (score >= 50)
{   printf("Passable\n");   }
if (score >= 75) // ✗ Cette condition est aussi vraie, le message s'affichera aussi !
{   printf("Bien\n");      }
if (score >= 90) // ✗ Toujours testé même si un autre `if` est déjà exécuté
{   printf("Excellent\n"); }
```

Ici, deux ou trois messages peuvent être affichés, ce qui n'est pas voulu.

Avec else if, un seul message sera affiché :

```
if (score >= 90)      {   printf("Excellent\n"); }
else if (score >= 75) {   printf("Bien\n");      }
else if (score >= 50) {   printf("Passable\n"); }
else                  {   printf("Échec\n");     }
```

Meilleure optimisation par le compilateur

- Certains compilateurs peuvent optimiser une structure "else if" pour qu'elle s'exécute plus rapidement qu'une série de "if" indépendants.

3.4 Risques et inconvénients:

Lisibilité et complexité accrue

Trop de "else if" rendent le code difficile à comprendre et à maintenir.

Un grand nombre de conditions empilées peut causer des erreurs et compliquer le débogage.

Risque d'oubli d'un cas

Avec de nombreux "else if", il est facile d'oublier un cas particulier.

Si aucune clause "else" n'est ajoutée, certaines valeurs peuvent ne pas être traitées.

Exemple avec un cas non couvert :

```
int note = 80;
```

```
if (note > 90)      {    printf("Excellent\n");    }
else if (note > 75) {    printf("Bien\n");          }
```

// ❌ Pas de else pour les autres cas

Solution : Ajouter toujours un "else" :

```
else                {    printf("Note non catégorisée\n"); }
```

Problèmes de performances

Chaque "else if" est évalué séquentiellement, ce qui peut ralentir l'exécution en cas de nombreuses conditions.

Dans certains cas, l'utilisation d'un **switch** est plus optimisée.

Exemple inefficace avec plusieurs else if :

```
if (valeur == 1)      {   printf("Un\n");   }
else if (valeur == 2) {   printf("Deux\n"); }
else if (valeur == 3) {   printf("Trois\n"); }
else                  {   printf("Autre\n"); }
```

Solution : Utiliser un switch qui est plus rapide dans certains cas :

```
switch (valeur)
{
    case 1: printf("Un\n");   break;
    case 2: printf("Deux\n"); break;
    case 3: printf("Trois\n"); break;
    default: printf("Autre\n"); break;
}
```

Avantage : En C, le switch peut être optimisé par le compilateur

Erreurs d'ordre des conditions

Les conditions doivent être placées dans le bon ordre, sinon certaines ne seront jamais atteintes.

Exemple incorrect :

```
if (score >= 50)
{   printf("Passable\n");   }
else if (score >= 90)        // ✗ Cette condition ne sera jamais atteinte
{   printf("Excellent\n");   }
```

Solution : Mettre les conditions les plus restrictives en premier.

3.5 Ce qu'il faut retenir:

Les structures "if else if" et "if if" sont plus ou moins identiques, mais il y a une différence subtile : dans le premier cas, une seule des deux instructions peut être exécutée tandis que dans le second cas, il est possible que les deux soient exécutées.

Cela peut devenir important s'il est possible que les deux conditions soient vraies. Dans ce cas, « if else if » n'exécuterait que la première instruction « if » et « if if » les exécuterait toutes les deux dans l'ordre.

Dans le cas des "else if" si le premier if est vrai, tous les autres ifs ne seront pas exécutés, même s'ils sont évalués comme vrais.

S'il s'agissait de ifs individuels, tous les ifs seront néanmoins exécutés s'ils sont évalués comme vrais.

Bonnes pratiques

L'utilisation de "else if" en C peut être dangereuse si elle est mal maîtrisée. Pour éviter les problèmes :

1. **Limiter le nombre de else if** : Trop de conditions imbriquées peuvent rendre le code difficile à lire.
2. **Utiliser switch/case si applicable** : Dans certains langages comme C, Java et JavaScript, si l'on compare une seule variable à plusieurs valeurs, un switch est plus efficace.
3. **Toujours prévoir un else** : Cela permet d'éviter les cas non traités.
4. **Prioriser les conditions les plus probables en premier** : Cela améliore la performance du programme.